Dockerfile: creación de imaxes personalizadas (de 0 a avanzado)

Referencia oficial: https://docs.docker.com/engine/reference/builder/

---

1. Que é un Dockerfile

Un Dockerfile é un ficheiro de texto que define paso a paso como crear unha imaxe personalizada de Docker. Cada liña contén unha instrución, e Docker executará esas instrucións de arriba abaixo para construír unha imaxe.

Cando executas:

```
docker build -t nome_imaxe .
```

Docker le o Dockerfile, executa cada paso e garda o resultado en capas reutilizables.

---

2. Primeiros exemplos: “Hello World”

2.1. Empregando sh

Imaxe que simplemente imprime unha mensaxe por pantalla.

```
FROM alpine:3.20
CMD ["sh", "-c", "echo 'Ola mundo desde Docker con Alpine!'"]
```

Construción e execución:

```
docker build -t ola-alpine .
docker run --rm ola-alpine
```

2.2. "Hello World" en Python

Este é o exemplo máis sinxelo posible: unha imaxe que simplemente imprime unha mensaxe.

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
```

```
COPY app.py .

CMD ["python", "app.py"]
```

app.py

```
print("Ola, Docker! Isto é o meu primeiro contedor.")
```

Construción e execución

```
docker build -t ola:v1 .
docker run --rm ola:v1
```

---

3. Exemplo básico con dependencias (requirements.txt)

Agora engadimos dependencias de Python (por exemplo Flask) e executamos un pequeno servidor web.

Estrutura do proxecto

```
/proyecto
├── Dockerfile
├── app.py
└── requirements.txt
```

requirements.txt

```
Flask==3.0.0
gunicorn==23.0.0
```

app.py

```
import os
from flask import Flask
app = Flask(__name__)

@app.route('/')
def inicio():
    return "Ola mundo desde Flask e Docker!"

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", "5000")))
```

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["python", "app.py"]
```

Comandos

```
docker build -t flask-basic:v1 .
docker run -d --name flask-basic -p 5000:5000 flask-basic:v1
curl --fail --retry 10 --retry-connrefused --retry-delay 1 http://localhost:5000/
docker stop flask-basic
docker rm flask-basic
```

---

4. Como funciona o proceso de construción

1. Docker le o Dockerfile de arriba a abaixo.
2. Instrucións como `RUN` e `COPY` crean capas de cambios no sistema de ficheiros; outras, como `CMD`, `ENV` e `EXPOSE`, configuran metadatos. `FROM` inicia unha etapa a partir dunha imaxe base.
3. As capas quedan gardadas en caché para acelerar futuras construcións.
4. O resultado é unha imaxe lista para crear contedores.

Se cambias unha liña superior (por exemplo un RUN), a caché das seguintes capas invalídase.

---

5. Sentenzas máis habituais

Instrución	Descrición	Exemplo
FROM	Imaxe base	FROM ubuntu:22.04
RUN	Executa comandos no proceso de build	RUN apt-get install -y python3
COPY	Copia ficheiros do host ao contedor	COPY . /app
WORKDIR	Define o directorio de traballo	WORKDIR /app
ENV	Define variables de entorno	ENV PORT=5000
EXPOSE	Documenta o porto usado pola app	EXPOSE 8080

Instrución	Descrición	Exemplo
USER	Define o usuario de execución	USER www-data
CMD	Comando por defecto ao iniciar	CMD ["python", "app.py"]
ENTRYPOINT	Comando fixo principal	ENTRYPOINT ["python"]
ARG	Variable dispoñible durante o build	ARG VERSION=1.0
HEALTHCHECK	Proba periódica de saúde	HEALTHCHECK CMD curl -f http://localhost:5000/

6. Scripts de inicio (start.sh)

Pódese empregar un script de shell para iniciar servizos ou facer tarefas previas antes de lanzar a aplicación.

start.sh

```
#!/usr/bin/env bash
set -e
echo "Iniciando aplicación..."
exec "$@"
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

RUN chmod +x start.sh
ENTRYPOINT ["./start.sh"]
CMD ["python", "app.py"]
```

7. Multi-stage build (optimización e tamaño reducido)

```
# syntax=docker/dockerfile:1
FROM python:3.12 AS builder
WORKDIR /wheels
COPY requirements.txt .
RUN pip wheel --wheel-dir=/wheels -r requirements.txt

FROM python:3.12-slim
```

```
WORKDIR /app
RUN --mount=type=bind,from=builder,source=/wheels,target=/wheels \
    pip install --no-cache-dir --no-index --find-links=/wheels -r
/wheels/requirements.txt
COPY . .
CMD ["python", "app.py"]
```

O directorio de wheels móntase só durante ese `RUN`: non queda almacenado na imaxe final. Esta imaxe exclúe as ferramentas da etapa `builder`; o tamaño respecto doutra imaxe baseada en `slim` debe medirse, non darse por garantido.

---

8. Usuario non root (boa práctica)

```
FROM python:3.12-slim

RUN adduser --system appuser
WORKDIR /app
COPY . .

RUN pip install --no-cache-dir -r requirements.txt
USER appuser

CMD ["python", "app.py"]
```

---

9. HEALTHCHECK e variables de entorno

```
FROM python:3.12-slim

ENV PORT=5000

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt

EXPOSE $PORT
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import os, urllib.request;
urllib.request.urlopen('http://127.0.0.1:' + os.environ['PORT'] + '/',
timeout=2).read()" || exit 1

CMD ["python", "app.py"]
```

Este exemplo reutiliza o `app.py` do apartado 3, que le `PORT`. A comprobación usa a biblioteca estándar de Python e non precisa instalar `curl` na imaxe. `EXPOSE` documenta o porto; para publicalo hai que usar `-p`:

```
docker build -t flask-health:v1 .
docker run -d --name flask-health -e PORT=5050 -p 5050:5050 flask-health:v1
curl --fail --retry 10 --retry-connrefused --retry-delay 1 http://localhost:5050/
# Agarda pola primeira comprobación de saúde antes de inspeccionar o estado.
docker inspect --format '{{.State.Health.Status}}' flask-health
# Tras a primeira comprobación correcta: healthy
docker stop flask-health
docker rm flask-health
```

---

10. .dockerignore

```
__pycache__/
*.pyc
.venv/
.git/
*.log
```

Evita copiar ficheiros innecesarios ao contedor, reducindo o tamaño da imaxe.

---

11. Comandos útiles de Docker

```
docker build -t app:v1 .
docker build -t app:v2 --no-cache .
docker run --rm -p 8080:5000 app:v1
docker run -it app:v1 /bin/bash
docker logs -f flask-basic
```

`docker logs` require o nome ou ID dun contedor existente; por exemplo, `flask-basic` mentres se executa o exemplo do apartado 3.

---

12. Mellores prácticas

- Usa imaxes oficiais e versionadas (por exemplo python:3.12-slim).
- Evita instalar paquetes innecesarios.
- Agrupa instalacións en poucas capas.
- Emprega usuario non root.
- Define HEALTHCHECK.
- Usa .dockerignore.
- Reutiliza caché instalando dependencias antes de copiar o código.

---

13. Fluxo completo de desenvolvemento e produción

```
# Desenvolvemento
docker build -t webapp:dev .
docker run --rm -p 5000:5000 -v "$PWD":/app webapp:dev

# Produción
docker build -t webapp:prod --pull .
docker run -d --name webapp-prod -p 8000:5000 webapp:prod gunicorn --bind
0.0.0.0:5000 --workers 2 app:app
curl --fail --retry 10 --retry-connrefused --retry-delay 1 http://localhost:8000/
docker logs webapp-prod
docker stop webapp-prod
docker rm webapp-prod
```

Estes comandos reutilizan a aplicación e as dependencias do apartado 3. En desenvolvemento execútase `python app.py`; en produción, Gunicorn serve a aplicación no porto 5000 do contedor, publicado no 8000 do host. Se empregas o Dockerfile do apartado 9, mantén `PORT=5000` para que coincida co porto de Gunicorn e co `HEALTHCHECK`.